Types of Errors

1 جدول المقارنة

النوع	المفهوم	الاحتمالية	التأثير/العلاقة
Single-Bit Error	تغيّر **بت واحد** فقط (0→1 أو 1→0)	**منخفض** — نادر في Serial Transmission	يُغيّر المعنى تماماً (مثال: STX→LF)
Burst Error	تغيّر **بتّين أو أكثر** (غير شرط التتابع)	**عالٍ** — شائع جداً	يتناسب طردياً مع **السرعة** و**مدة التشويش**

2 تفاصيل النوعين

① Single-Bit Error

- يُصيب **بتّاً واحداً** فقط في وحدة البيانات (بايت أو حزمة)
- مثال: `00000010` → `00001010` (تغيّر البت الرابع = اختلاف المعنى كاملاً)
- نادر لأن **مدة التشويش** (Noise) عادةً **أطول** من زمن إرسال بت واحد (ميكروثانية عند 1 Mbps)

② Burst Error

- يُصيب **بتّين أو أكثر** — لا يشترط التجاور
- **طول الخطأ** = المسافة من أول بت تالف لآخر بت تالف (قد توجد بتات سليمة بينهما)
- شائع لأن التشويش يستمر لفترة تُصيب **مجموعة كاملة** من البتات

3 تنبيه ذكي — السرعة vs عدد البتات التالفة

سرعة النقل	مدة التشويش	عدد البتات التالفة
1 Kbps	1/100 ثانية	10 bits
1 Mbps	1/100 ثانية	10,000 bits

> ⚠ الخلاصة: نفس مدة التشوishtish تُفسد كمية أكبر من البتات عند زيادة السرعة.
> العلاقة طردية: كلما زادت Data Rate، زاد عدد البتات المُفسَدة لنفس مدة الخطأ.

Error Detection & Correction Principles

1 مبدأ التكرار (Redundancy)

- المفهوم المحوري = إضافة Redundant bits (بتات إضافية) للبيانات الأصلية
- المُرسِل يُضيفها ← المستقبل يستخدمها للتحقق ← ثم يزيلها
- الهدف: اكتشاف التلف (Corruption) أثناء النقل

---

2 اكتشاف vs تصحيح — المقارنة الحاسمة

الجانب	Error Detection	Error Correction
الهدف	معرفة وجود خطأ (نعم/لا)	تحديد **الموقع** الدقيق والعدد
المرونة	لا يهم عدد الأخطاء	يجب تحديد **أي بت** تالف لعكسه
السهولة	بسيط نسبياً	معقد ويحتاج بتات أكثر
النتيجة	إعلان بوجود خطأ	إصلاح فوري للخطأ

3 آليات التصحيح الرئيسية

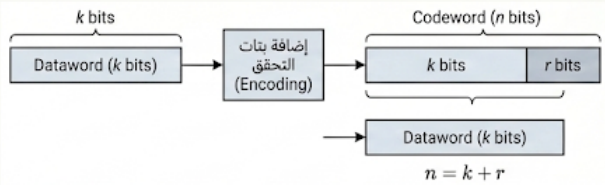
- Forward Error Correction (FEC) ← المستقبل يُصحح تلقائياً باستخدام البتات الإضافية (للأخطاء القليلة)
- Retransmission ← المستقبل يكتشف ويطلب إعادة الإرسال من المُرسِل حتى تصل صحيحة

---

4 . تشفير الكتل (Block Coding)

المفهوم	الرمز	التعريف	الحجم
Dataword	k	كتلة البيانات الأصلية	k bits
Codeword	n	الكلمة المشفرة بعد إضافة البتات	$n = k + r$
Redundant bits	r	البتات الإضافية	r bits

- **العلاقة الأساسية:** المعادلة التي تحكم حجم الكلمة المشفرة هي: $n = k + r$
- **عدد الكلمات:** بما أننا أضفنا بتات زائدة، فإن عدد الكلمات المشفرة المحتملة هو **دائماً أكبر** من عدد كلمات البيانات (2^k):
  $2^n > 2^k$
- **الكلمات غير الصالحة (Invalid Codewords):** هي تلك الكلمات الناتجة عن بين 2^n و 2^k؛ حيث تُستخدم الكلمات الزائدة للكشف عن وجود خطأ عند الاستلام.



5 قواعد الحساب في نظام Modulo-2 (القواعد الذهبية)

- **نظام ثنائي:** يتعامل هذا النظام مع الرقمين (0 و 1) فقط.
- **تماثل العمليات:** في حساب Modulo-2، تعتبر عملية الجمع هي نفسها عملية الطرح، وكلاهما يُنفذ عبر منطق XOR.

العملية الحسابية	النتيجة	التوضيح
$0+0$	**0**	بتات متشابهة
$1+1$	**0**	بتات متشابهة
$1+0$	**1**	بتات مختلفة
$0+1$	**1**	بتات مختلفة

⚙️ **القاعدة العامة:**
✅ * إذا كان **البتان متشابهان** (Same) ← النتيجة دائماً **0**.
ℹ️ * إذا كان **البتان مختلفان** (Different) ← النتيجة دائماً **1**.

6 مسافة هامينج (Hamming Distance)

التعريف: عدد الاختلافات بين البتات المتقابلة في كلمتين متساويتي الحجم.

الحد الأدنى لمسافة هامينج: **(d_{min}):** أصغر مسافة بين أي زوج من الكلمات المشفرة الصالحة.

معادلات التصميم:

1. **لاكتشاف s من الأخطاء:**
   يجب أن تكون المسافة كافية بحيث لا يتحول الرمز الأصلي إلى رمز صالح آخر.

$$d_{min} \geq s + 1$$

2. **لتصحيح t من الأخطاء:**
   يجب أن يظل الرمز المشوه أقرب إلى الرمز الصحيح من أي رمز آخر.

$$d_{min} \geq 2t + 1$$

مثال سريع للتوضيح:

- إذا كانت $d_{min} = 3$:
  - يمكنك **اكتشاف** خطأين $(s = 2)$ لأن $3 \geq 2 + 1$.
  - يمكنك **تصحيح** خطأ واحد فقط $(t = 1)$ لأن $3 \geq 2(1) + 1$.

اكتشاف وتصحيح الأخطاء

◆ 1. مفهوم التكرار وتشفير الكتل (Block Coding)

تعتمد الحماية على إضافة بتات حشو (Redundancy) لتحويل كلمة البيانات إلى كلمة مشفرة.

- **المعادلة الأساسية:**

$$n = k + r$$

- k: بتات البيانات الأصلية (Dataword).
- r: بتات التكرار المضافة (Redundant bits).
- n: الطول الكلي للكلمة المشفرة (Codeword).

- **الكلمات غير الصالحة (Invalid):** هي الفارق بين عدد الكلمات الممكنة والكلمات الصالحة ($2^n - 2^k$)؛ إذا استلمها الجهاز يكتشف الخطأ فوراً.

◆ 2. مسافة هامينج (Hamming Distance) والقواعد الذهبية

هي مقياس الاختلاف بين كلمتين باستخدام بوابة XOR. أهم معيار للتصميم هو الحد الأدنى للمسافة d_{min}.

الهدف من النظام	القاعدة الرياضية	ملاحظة امتحانية
اكتشاف s من الأخطاء	$d_{min} = s + 1$	تضمن عدم تحول كلمة صالحة لأخرى صالحة مصادفة.
تصحيح t من الأخطاء	$d_{min} = 2t + 1$	تحتاج مسافة أكبر لبناء "منطقة تخمين" حول الكلمة.

◆ 3. شفرات هامينج (Hamming Codes)

تطبيق لتقنية **FEC**، وتصمم دائماً بمسافة

$$d_{min} = 3$$

(تصحيح خطأ واحد أو اكتشاف خطأين).

- **المتغير** m: يجب أن يكون أكبر من أو يساوي 3.
- **طول الكلمة الكلي** (n):

$$n = 2^m - 1$$

.

- **طول البيانات الصافي** (k):

$$k = n - m$$

.

- **مثال شهير**: نظام $C(7, 4)$ حيث $n = 7$ و $k = 4$.

◆ 4. فحص التكرار الدوري (CRC)

التقنية الأقوى لكشف **أخطاء الحزمة (Burst Errors)** باستخدام حسابات المقياس 2 وقسمة كثيرات الحدود.

احتمالات النجاح والفشل في كشف الخطأ:

- **طول الخطأ** (L) **أصغر من أو يساوي** r: اكتشاف بنسبة **100%**.
- **طول الخطأ** (L) **يساوي** $r + 1$: احتمال الفشل هو

$$(1/2)^{r-1}$$

.

- **طول الخطأ** (L) **أكبر من** $r + 1$: احتمال الفشل هو

$$(1/2)^r$$

.

🔷 5. المجموع الاختباري (Checksum)

يستخدم في بروتوكولات الإنترنت ويعتمد على "المتمم الأحادي" (**One's Complement**).

- **آلية العمل:** تقسيم الرسالة لـ 16 بت وجمعها.
- **اللف (Wrapping):** جمع البتات الفائضة (Carry) من اليسار مع الناتج مرة أخرى.
- **النتيجة:** تُعكس القيم (0 يصبح 1 والعكس) لتُرسل كقيمة سالبة؛ وإذا كان مجموع المستقبل (بيانات + Checksum) يساوي أصفاراً، تُقبل الرسالة.

📝 **بنك أسئلة MCQ (للمحترفين)**

Q1: إذا كان طول كلمة البيانات $k = 5$ وطول الكلمة المشفرة $n = 8$، فكم عدد الكلمات "غير الصالحة"؟

أ) 3

ب) 32

ج) 224 $(2^8 - 2^5)$ ✅

د) 256

Q2: لتصحيح خطأين ($t = 2$) في نظام ما، يجب أن تكون d_{min} تساوي على الأقل:

أ) 3

ب) 4

ج) 5 ✅

د) 2

Q3: في شفرة هامينج، إذا كان $m = 4$، فما هو طول البيانات الصافي k؟

أ) 15

ب) 11 ✅ $(15 - 4)$

ج) 7

د) 4

Q4: أي تقنية تعتمد على "قسمة كثيرات الحدود" لاكتشاف أخطاء الحزمة؟

أ) Checksum

ب) Parity Check

ج) CRC ✅

د) Hamming Code

Q5: في الـ Checksum، إذا نتج عن الجمع بت فائض من جهة اليسار، نقوم بـ:

أ) تجاهله تماماً

ب) حذفه وعكس البيانات

ج) جمعه مع الناتج (Wrapping) ✅

د) إيقاف عملية الإرسال

Error Correction Strategies

1 لماذا التصحيح أصعب من الاكتشاف؟

- **الاكتشاف (Detection):** يسأل فقط "هل حدث خطأ؟" (الإجابة نعم/لا) ← عملية بسيطة.
- **التصحيح (Correction):** يسأل "أين الخطأ؟ وكم عدده؟" ← يتطلب:
  1. تحديد العدد الدقيق للبتات التالفة.
  2. تحديد **الموقع الدقيق** (Exact Location) لكل بت تالف.
  3. عكس قيمة البت (0 ← 1 أو 1 ← 0).
- **النتيجة:** التصحيح يحتاج بتات حشو (Redundant bits) أكثر بكثير من الاكتشاف.

2 مقارنة الاستراتيجيتين الرئيسيتين

الجانب	تصحيح الخطأ الأمامي (FEC)	إعادة الإرسال (Retransmission)
من يُصحح؟	المُستقبِل ذاتياً (Self-Correction)	المُرسِل (بعد طلب من المستقبل)
الآلية	استنتاج الرسالة من الـ Redundant bits	اكتشاف الخطأ ← رفض الطرد ← طلب إعادة الإرسال
المتطلب الرياضي	$d_{min} \geq 2t + 1$	طريقة اكتشاف قوية (مثل CRC)
متى تُستخدم؟	البث أحادي الاتجاه / تأخير القناة عالٍ	شبكات ثنائية الاتجاه (Internet)
العيوب	تستهلك Bandwidth إضافي دائم للـ Check bits	تسبب تأخيراً (Delay) عند كثرة الأخطاء
أمثلة	Hamming Codes	بروتوكول TCP / نظام ARQ

3 مفهوم النطاق (Territory) في FEC

تخيل أن كل كلمة مشفرة صالحة (Valid Codeword) هي مركز لدائرة (كرة هامينج):

- **نصف القطر (t):** يمثل عدد الأخطاء التي يمكن للمستقبل تصحيحها.
- **الآلية:** أي كلمة غير صالحة تسقط داخل نطاق الدائرة، "يسحبها" المستقبل إلى المركز (الكلمة الصالحة الأقرب).
- **الشرط الجوهري:** لكي لا يخطئ المستقبل في التخمين، يجب ألا تتداخل الدوائر أبداً، وهذا يتحقق عندما تكون المسافة بين المركزين:

$$d_{min} \geq 2t + 1$$

5 Hamming Codes — المثال الأبرز

- تعتمد على $d_{min} = 3$.
- **القدرة:** تصحيح **خطأ واحد فقط** (Single-Bit Error) في كل كتلة بيانات.
- **الآلية:** توليد "متلازمات" (Syndromes) عبر عمليات XOR؛ إذا كانت قيمة السندروم لا تساوي صفراً، فهي تشير مباشرة إلى رقم البت التالف لقلبه.